Comparison of various U-Net Models on prostate segmentation task

Introduction about task

Using U-Net segmentation models for prostate segmentation involves the application of a specific type of deep learning architecture called U-Net to accurately and automatically identify and segment the prostate gland in medical images. The U-Net architecture is particularly well-suited for this task and has been widely utilized in medical imaging due to its effectiveness. U-Net is a convolutional neural network (CNN) architecture that consists of an encoding path and a decoding path. The encoding path gradually reduces the spatial dimensions of the input image while capturing high-level features. This is achieved through a series of convolutional and pooling layers. The decoding path then upsamples the encoded features and recovers the spatial resolution using transpose convolutions, allowing for precise localization. Prostate segmentation using U-Net models typically involves training the network on a large dataset of annotated prostate images. The model learns to map input images to pixel-wise segmentation masks, which indicate the regions belonging to the prostate gland. The training process involves optimizing the model's parameters to minimize the discrepancy between the predicted segmentation and the ground truth annotations.

Once trained, the U-Net model can be applied to unseen medical images to automatically segment the prostate gland. The model analyzes the image and generates a segmentation mask, delineating the boundaries of the prostate. This automated segmentation process reduces the need for manual intervention and improves efficiency in clinical workflows. In fact, The use of U-Net segmentation models for prostate segmentation offers several advantages. It enables accurate and consistent segmentation, reducing the reliance on manual annotations, which can be time-consuming and subjective. U-Net models can capture intricate details and complex spatial relationships within the prostate gland, improving the accuracy of segmentation compared to traditional methods. Furthermore, U-Net models are flexible and can be adapted to different imaging modalities, such as magnetic resonance imaging (MRI) or computed tomography (CT), making them suitable for diverse clinical scenarios. The availability of reliable and automated prostate segmentation can enhance diagnosis, treatment planning, and monitoring of prostate-related diseases, leading to improved patient outcomes.

However, it is important to note that while U-Net models have demonstrated promising results, they are not without limitations. The performance of the models heavily relies on the quality

and diversity of the training data, and the generalization to unseen images should be carefully evaluated. Ongoing research and collaboration between medical professionals and machine learning experts are essential to further refine and validate the use of U-Net segmentation models for prostate segmentation in clinical practice.

About dataset

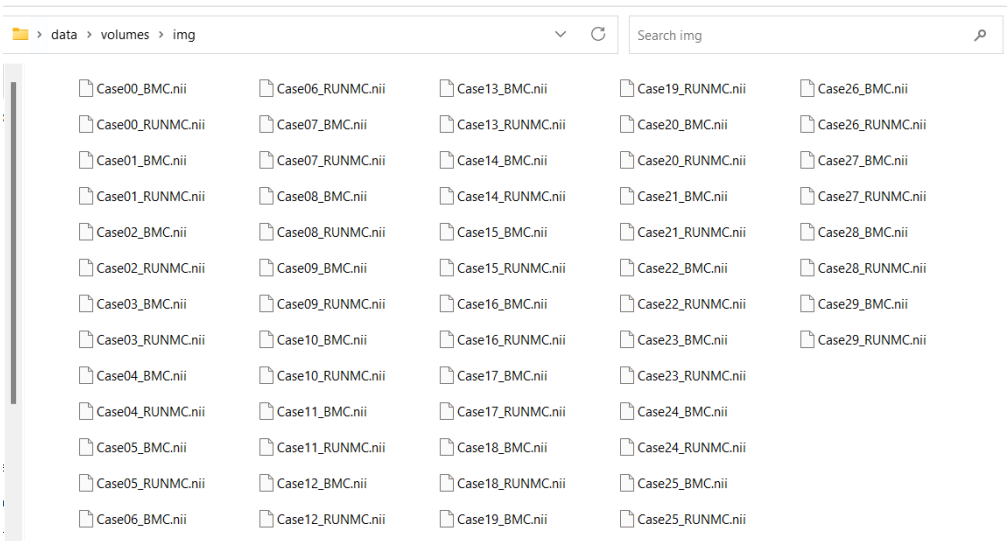
"A Multi-site Dataset for Prostate MRI Segmentation" refers to a dataset that has been curated specifically for the task of segmenting the prostate gland in magnetic resonance imaging (MRI) scans. This dataset is unique because it contains MRI scans from multiple medical centers or sites, rather than being limited to data from a single source. The development of robust and accurate segmentation algorithms for prostate MRI is challenging due to the anatomical variations between individuals and the differences in MRI acquisition protocols across different sites. By using a multi-site dataset, researchers can better capture the diversity of patient populations and imaging protocols, making the segmentation models more robust and generalizable.

The dataset typically includes many MRI scans acquired from multiple medical centres, representing a broad range of patient demographics and imaging characteristics. Each MRI scan is accompanied by corresponding manual annotations or ground truth segmentations, where experts have delineated the prostate gland boundaries.

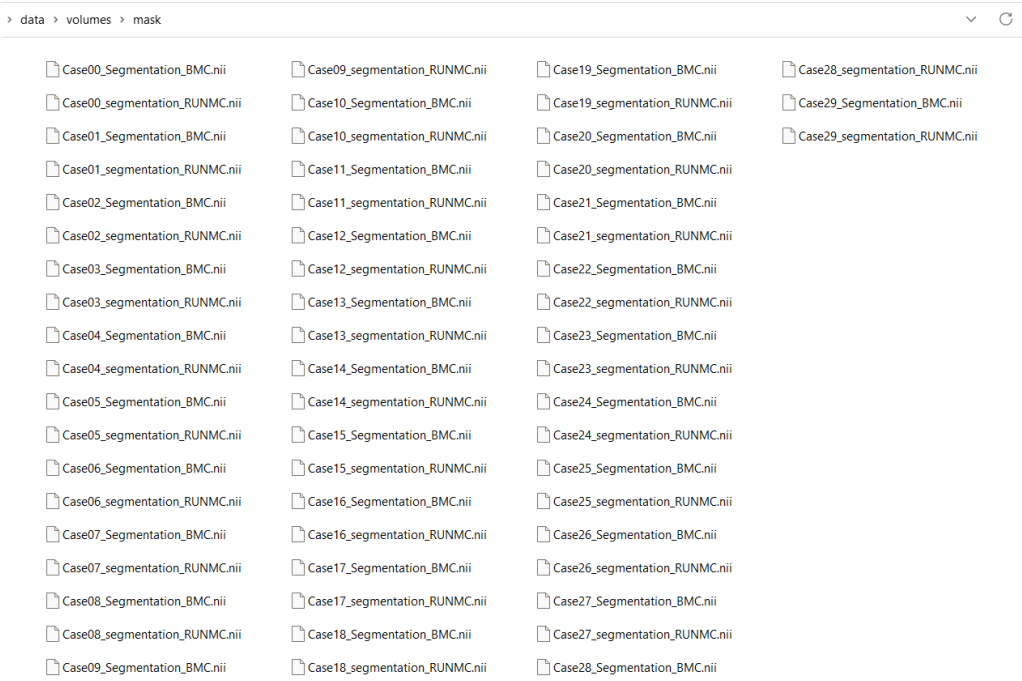
The dataset is composed of 6 sites, discussed with details in below:

Dataset	Institution	Case num	Field strength(T)	Resolution(in/ through plane)(mm)	Endorectal Coil	Manufactor
Site A	RUNMC	30	3	0.6-0.625/3.6-4	Surface	Siemens
Site B	BMC	30	1.5	0.4/3	Endorectal	Philips
Site C	HCRUDB	19	3	0.67-0.79/1.25	No	Siemens
Site D	UCL	13	1.5 and 3	0.325-0.625/3-3.6	No	Siemens
Site E	BIDMC	12	3	0.25/2.2-3	Endorectal	GE
Site F	HK	12	1.5	0.625/3.6	Endorectal	Siemens

We have used two of them, two sites, A (RUNMC) and B (BMC) including (.nii) format files in both images and their counterpart masks from 30 cases, MRI format of images shown :



and their counterpart masks:



Experimental Results

Step 1) Pre-processing:

- **Installing and importing necessary packages and libraries**

In [1]:
```
# installing necessary packages

# for loading and reading .nii format
!pip install nibabel
!pip install opencv-python
!pip install scikit-learn
```

```
Requirement already satisfied: nibabel in c:\users\behno\anaconda3\lib\site-packages (5.1.0)
Requirement already satisfied: packaging>=17 in c:\users\behno\anaconda3\lib\site-packages (from nibabel) (22.0)
Requirement already satisfied: numpy>=1.19 in c:\users\behno\anaconda3\lib\site-packages (from nibabel) (1.23.5)
Requirement already satisfied: opencv-python in c:\users\behno\anaconda3\lib\site-packages (4.7.0.72)
Requirement already satisfied: numpy>=1.17.0 in c:\users\behno\anaconda3\lib\site-packages (from opencv-python) (1.23.5)
Requirement already satisfied: scikit-learn in c:\users\behno\anaconda3\lib\site-packages (1.2.1)
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\users\behno\anaconda3\lib\site-packages (from scikit-learn) (2.2.0)
Requirement already satisfied: numpy>=1.17.3 in c:\users\behno\anaconda3\lib\site-packages (from scikit-learn) (1.23.5)
Requirement already satisfied: joblib>=1.1.1 in c:\users\behno\anaconda3\lib\site-packages (from scikit-learn) (1.1.1)
Requirement already satisfied: scipy>=1.3.2 in c:\users\behno\anaconda3\lib\site-packages (from scikit-learn) (1.10.0)
```

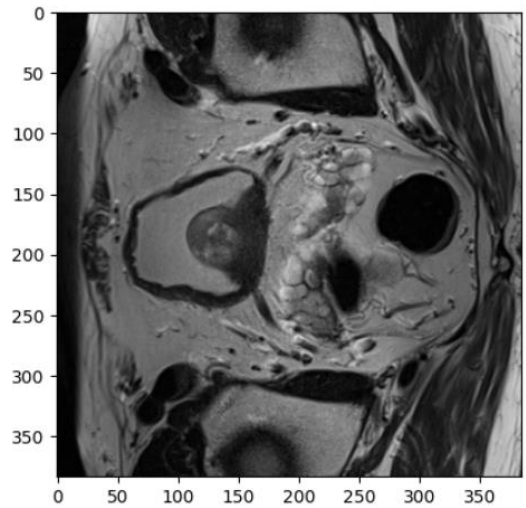
In [2]:
```
# importing libraries
import os, glob
import nibabel as nib
import numpy as np
import matplotlib.pyplot as plt
import cv2
```

- **Hounsfield units (HU)**

Hounsfield units (HU) are a measurement scale used in computed tomography (CT) imaging to quantify the radiodensity of tissues. The scale is based on the linear attenuation coefficients of tissues relative to that of water. Water is assigned a value of 0 HU, and the scale extends both above and below that point. In CT imaging, different tissues have different Hounsfield unit values. For example, air has a value of approximately -1000 HU, while bone can range from +1000 HU to +3000 HU, depending on the specific bone and its density. Soft tissues like muscle and organs usually have HU values in the range of +10 HU to +60 HU.

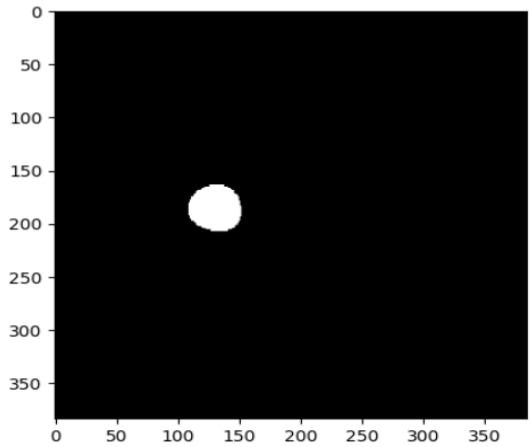
The Hounsfield unit scale allows for the differentiation and visualization of various tissues within a CT image. It helps in distinguishing between different densities and provides information about the composition and characteristics of the imaged structures. However, it's important to note that MRI imaging does not use Hounsfield Units. In MRI, images are typically represented using grayscale intensity values that reflect the relaxation times of protons within tissues. The interpretation of MRI images relies on recognizing the specific signal characteristics of different tissues rather than using a numerical scale like Hounsfield Units in CT imaging. To process our .nii format files in our neural network structur, as they are based on (hight*width* depth) format, with static hight and width 328 and different depths (like 32,20, 27, and …), meaning that they have different volumes. Thus, we have extracted each volume size as one slice. For example, the .nii file with hight and width 328 and depth 27, creates 27 images, which is constructed by same hight and width, 328. As it is shown below, you can see .nii volume-based image and their counterpart mask in specific slice (slice 20) :

```
+ Code   + Text

[9]
# Load image and see max min Hounsfield units
imagePathInput="/content/drive/MyDrive/prostat segmentation/data/volumes/img"
imgPath = os.path.join(imagePathInput, 'Case02_RUNMC.nii')
img = nib.load(imgPath).get_fdata()
np.min(img), np.max(img), img.shape, type(img)

(0.0, 1967.5, (384, 384, 24), numpy.memmap)

# Show image slice
imgSlice = img[:,:,20]
plt.imshow(imgSlice, cmap='gray')
plt.show()
```



Slice image

```
+ Code  + Text

[18] # Load mask and see max min value of it
     imagePathInput="/content/drive/MyDrive/prostat segmentation/data/volumes/mask"
     imgPath = os.path.join(imagePathInput, 'Case02_segmentation_RUNMC.nii')
     mask = nib.load(imgPath).get_fdata()
     np.min(mask), np.max(mask), mask.shape, type(mask)

     (0.0, 2.0, (384, 384, 24), numpy.memmap)

[21] # Show mask slice
     imgSlice = mask[:,:,20]
     plt.imshow(imgSlice, cmap='gray')
     plt.show()
```



Slice mask

- **Loading images and masks**

We should define the 4 folders which contains, (.nii) masks and images, and (.png) mask and images, in 4 different paths, such as below:

```
# STEP 1 - Load and visualize data
# path of .nii format images and mask
dataInputPath = 'data/volumes/'
imagePathInput = os.path.join(dataInputPath, 'img/')
maskPathInput = os.path.join(dataInputPath, 'mask/')

# path of .png format images and mask
dataOutputPath = 'data/slices/'
imageSliceOutput = os.path.join(dataOutputPath, 'img/')
maskSliceOutput = os.path.join(dataOutputPath, 'mask/')
```

- **Normalizing images with pixel values between 0 to 1 (using MinMaxScalar)**

```python
# Normalize image
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()

def normalizeImageIntensityRange(img):
    img=scaler.fit_transform(img.reshape(-1, img.shape[-1])).reshape(img.shape)
    return img

nImg = normalizeImageIntensityRange(img)
np.min(nImg), np.max(nImg), nImg.shape, type(nImg)
```

```
(0.0, 1.0, (384, 384, 24), numpy.ndarray)
```

- **Functions to read .nii images and mask from paths and converting them to .png format in another paths**

```python
[15] # Read image or mask volume
def readImageVolume(imgPath, normalize=False):
    img = nib.load(imgPath).get_fdata()
    if normalize:
        return normalizeImageIntensityRange(img)
    else:
        return img
```

```python
[16] # Save volume slice to file
def saveSlice(img, fname, path):
    img = np.uint8(img * 255)
    fout = os.path.join(path, f'{fname}.png')
    cv2.imwrite(fout, img)
    print(f'[+] Slice saved: {fout}', end='\r')
```

```python
# Slice image in all directions and save the z axis (depth of Volume)
# STEP 2 - Slicing and saving
SLICE_Z = True
SLICE_DECIMATE_IDENTIFIER = 3

def sliceAndSaveVolumeImage(vol, fname, path):
    (dimx, dimy, dimz) = vol.shape

    print("dimz : "+str(dimz))

    cnt = 0
    if SLICE_Z:
        cnt += dimz
        print('Slicing Z: ')
        for i in range(dimz):
            saveSlice(vol[:,:,i], fname+f'-slice{str(i).zfill(SLICE_DECIMATE_IDENTIFIER)}_z', path)
    return cnt
```

- **Loading .nii images and mask from paths and converting them to .png format in another paths.**

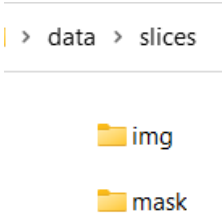
```python
# Read and process image volumes
for index, filename in enumerate(sorted(glob.iglob(imagePathInput+'*.nii'))):
    img = readImageVolume(filename, True)
    print(filename, img.shape, np.sum(img.shape), np.min(img), np.max(img))
    numOfSlices = sliceAndSaveVolumeImage(img, 'prostat'+str(index), imageSliceOutput)
    print(f'\n{filename}, {numOfSlices} slices created \n')
```

```
data/volumes/img/Case00_BMC.nii (384, 384, 32) 800 0.0 1.0
dimz : 32
Slicing Z:
[+] Slice saved: data/slices/img/prostat0-slice031_z.png
data/volumes/img/Case00_BMC.nii, 32 slices created

data/volumes/img/Case00_RUNMC.nii (384, 384, 15) 783 0.0 1.0
dimz : 15
Slicing Z:
[+] Slice saved: data/slices/img/prostat1-slice014_z.png
data/volumes/img/Case00_RUNMC.nii, 15 slices created

data/volumes/img/Case01_BMC.nii (384, 384, 30) 798 0.0 1.0
dimz : 30
Slicing Z:
[+] Slice saved: data/slices/img/prostat2-slice029_z.png
data/volumes/img/Case01_BMC.nii, 30 slices created

data/volumes/img/Case01_RUNMC.nii (384, 384, 20) 788 0.0 1.0
dimz : 20
Slicing Z:
```

```python
# Read and process image mask volumes
for index, filename in enumerate(sorted(glob.iglob(maskPathInput+'*.nii'))):
    img = readImageVolume(filename, False)
    print(filename, img.shape, np.sum(img.shape), np.min(img), np.max(img))
    numOfSlices = sliceAndSaveVolumeImage(img, 'segmented'+str(index), maskSliceOutput)
    print(f'\n{filename}, {numOfSlices} slices created \n')
```

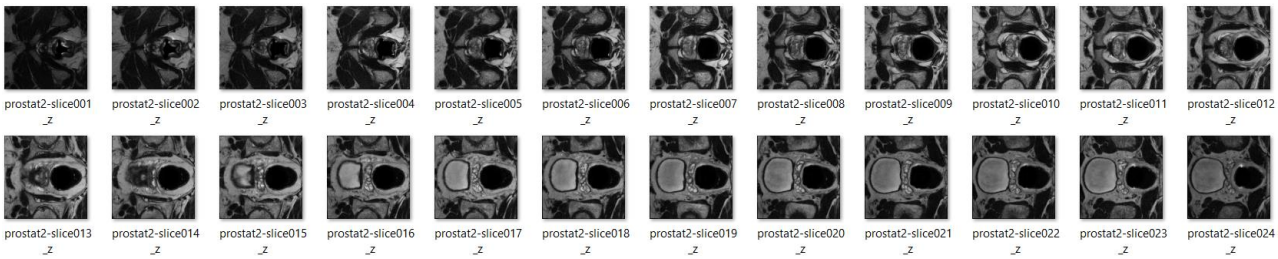
```
data/volumes/mask/Case00_Segmentation_BMC.nii (384, 384, 32) 800 0.0 2.0
dimz : 32
Slicing Z:
[+] Slice saved: data/slices/mask/segmented0-slice031_z.png
data/volumes/mask/Case00_Segmentation_BMC.nii, 32 slices created

data/volumes/mask/Case00_segmentation_RUNMC.nii (384, 384, 15) 783 0.0 2.0
dimz : 15
Slicing Z:
```

Consequently, We converted our dataset (each extracted images form depth) into .png format (images and masks) used in our learning procedure as training and validation dataset. They have same size in both hight and width 328*328 and one channel as Gray-scale image with pixel values between 0-255. however, we have binary mask images including 0 or 1 value with respect to area. As you can observe, we have two different folders, which contains our images and masks (.png format):



Dataset folders

Two figures below show the input image and their masks:



.png images

.png masks

Step 2) Evaluating different Segmentation Models:

- **Installing and importing necessary packages and libraries**

In [1]:

```
# importing libraries
import os, glob
os.environ["SM_FRAMEWORK"] = "tf.keras"
import nibabel as nib
import numpy as np
import matplotlib.pyplot as plt
import cv2
from keras.optimizers import Adam
from tensorflow import keras
import segmentation_models as sm
from keras.callbacks import ModelCheckpoint
from tensorflow import keras
```

```
2023-06-25 03:17:53.975498: I tensorflow/core/util/port.cc:110] oneDNN custom operations are on. You may see slightly different
numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environme
nt variable `TF_ENABLE_ONEDNN_OPTS=0`.
2023-06-25 03:17:54.004093: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use ava
ilable CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler
flags.
2023-06-25 03:17:54.388793: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not find TensorRT
```

```
Segmentation Models: using `tf.keras` framework.
```

- **Paths for reading our dataset (including images and masks)**

In [3]:
```
# paths for .png images and masks
#Capture training images and masks info as a list
train_images = []

for directory_path in glob.glob("/home/behnoud/Desktop/Desktop/data/slices/img2"):
    for img_path in sorted(glob.glob(os.path.join(directory_path, "*.png"))):
        img = cv2.imread(img_path, cv2.IMREAD_COLOR)
        train_images.append(img)

#Convert list to array for machine learning processing
train_images = np.array(train_images)

#Capture mask/label info as a list
train_masks = []
for directory_path in glob.glob("/home/behnoud/Desktop/Desktop/data/slices/mask2"):
    for mask_path in sorted(glob.glob(os.path.join(directory_path, "*.png"))):
        mask = cv2.imread(mask_path, 0)
        train_masks.append(mask)

#Convert list to array for machine learning processing
train_masks = np.array(train_masks)
```

- **Normalize input images and binarize the masks**

In [4]:
```
#Constructing image (input) and mask (output)
X = train_images
Y = train_masks.astype(float)
Y = np.expand_dims(Y, axis=3)

#binarize Y mask to two values 0 and 1
#as sum pixels we have more than two labels in some images, we assign them to label 1
Y [Y == 254.] = 255.
Y = Y/255.

print(X.shape)
print(Y.shape)
```

```
(160, 384, 384, 3)
(160, 384, 384, 1)
```

- **Choosing the backbones of UNet model and preprocessing input based on it**

In [2]:
```
# defining different backbones

BACKBONE = 'vgg16'
#BACKBONE = 'vgg19'
#BACKBONE = 'resnet18'
#BACKBONE = 'resnet34'
#BACKBONE = 'resnet50'
#BACKBONE = 'densenet121'
#BACKBONE = 'densenet169'
#BACKBONE = 'densenet201'
#BACKBONE = 'efficientnetb0'
#BACKBONE = 'efficientnetb1'
#BACKBONE = 'efficientnetb2'
#BACKBONE = 'efficientnetb3'
#BACKBONE = 'mobilenet'
#BACKBONE = 'mobilenetv2'

preprocess_input = sm.get_preprocessing(BACKBONE)

#Resize images (height  = X, width = Y)
SIZE_X = 384
SIZE_Y = 384
```

Segmentation models with different backbones

In the field of semantic segmentation, the U-Net architecture is widely used due to its effectiveness in capturing fine-grained details while maintaining a large receptive field. The backbone of a U-Net refers to the encoder network that is typically used as a feature extractor. There are several different backbones that can be used with U-Net, and each has its own characteristics. Here are some commonly used backbones in segmentation libraries:

1. VGG16/VGG19: VGG (Visual Geometry Group) is a popular convolutional neural network (CNN) architecture. VGG16 and VGG19 are deep models with 16 and 19 weight layers, respectively. They are known for their simplicity and uniform architecture, consisting of multiple stacked 3x3 convolutional layers with max pooling. VGG backbones provide a good trade-off between accuracy and computational efficiency.
2. ResNet (Residual Network): ResNet is a deep CNN architecture that introduces residual connections to address the vanishing gradient problem. The residual connections allow gradients to flow directly through the network, enabling the training of very deep networks (e.g., ResNet50, ResNet101, ResNet152). ResNet backbones are known for their excellent performance in various computer vision tasks, including semantic segmentation.
3. DenseNet: DenseNet is another CNN architecture that promotes feature reuse and encourages gradient flow throughout the network. It connects each layer to every other layer in a feed-forward fashion, resulting in densely connected blocks. DenseNet backbones have shown strong performance in segmentation tasks by effectively capturing multi-scale contextual information.
4. EfficientNet: EfficientNet is a family of CNN architectures designed with an emphasis on model efficiency and scaling. These models use a compound scaling method that optimizes the depth, width, and resolution of the network. EfficientNet backbones achieve high performance with fewer parameters, making them well-suited for resource-constrained environments.
5. MobileNetV2/MobileNetV3: MobileNet is a lightweight CNN architecture specifically designed for mobile and embedded devices. MobileNetV2 and MobileNetV3 are subsequent versions that improve upon the original MobileNet. They employ depthwise separable convolutions to reduce computational complexity while maintaining competitive accuracy. MobileNet backbones are commonly used when efficiency is a primary concern.

The choice of backbone depends on various factors such as the available computational resources, task requirements (e.g., accuracy, speed), and memory constraints. It is important to evaluate the performance of different backbones on the specific segmentation task and choose the one that best suits your needs.

- **Splitting dataset to train/validation/test**

In [5]:
```
# spilitting data to train and validation set
from sklearn.model_selection import train_test_split
x_train, x_val, y_train, y_val = train_test_split(X, Y, test_size=0.2, random_state=42)
x_test,_,y_test,_= train_test_split(x_val, y_val, test_size=0.5, random_state=42)

# preprocess input data
x_train = preprocess_input(x_train)
x_val = preprocess_input(x_val)

```

- **Creating the model and selecting loss function, optimizer, and metrics to be evaluated by model, and then compiling**

In [6]:
```
#create model and using sigmoid as we have binary classification in our output
model = sm.Unet(BACKBONE, classes=1, activation='sigmoid')

# define optomizer
LR= 0.0001
optim = keras.optimizers.Adam(LR)

# Segmentation models losses can be combined together by '+' and scaled by integer or float factor
dice_loss = sm.losses.DiceLoss()
focal_loss = sm.losses.BinaryFocalLoss()
total_loss = dice_loss + (1 * focal_loss)

metrics = [sm.metrics.IOUScore(threshold=0.5), sm.metrics.FScore(threshold=0.5)]

# compile keras model with defined optimozer, loss and metrics
model.compile(optim, total_loss, metrics)
```

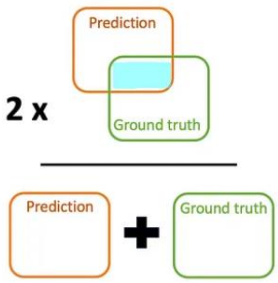
Loss function (or metrics):

The Dice Loss and Focal Loss are commonly used loss functions in the field of deep learning for various tasks, including segmentation and object detection. Here are the formulas for each loss function:

- Dice Loss:

The Dice Loss measures the similarity between two sets, usually the predicted and ground truth segmentation masks. It is defined as:

$$\text{Dice} = \frac{2\ X\ Area\ of\ overlap}{Total\ area} =$$



Dice Loss = 1 - Dice

where Intersection is the number of pixels common to both masks, and Union is the total number of pixels in both masks.

- Focal loss:

The Focal Loss is designed to address the class imbalance problem commonly encountered in tasks with highly imbalanced datasets. It introduces a modulating factor that down-weights the loss contribution from well-classified examples. The formula for the Focal Loss is:

$$\mathrm{FL}(p_{\mathrm{t}}) = -\alpha_{\mathrm{t}}(1 - p_{\mathrm{t}})^{\gamma} \log(p_{\mathrm{t}}).$$

where:

- α is the balancing factor that weights the contribution of positive and negative examples.
- p is the predicted probability of the positive class.
- γ is the focusing parameter that controls the rate at which easy examples are downweighted. A higher value of γ focuses more on hard examples.

Combining Dice Loss and Focal Loss:

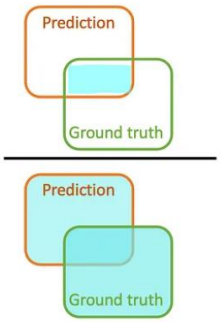
The combination of Dice Loss and Focal Loss is often used to leverage the benefits of both loss functions. One way to combine them is by simply summing or averaging their individual losses. For example, the combined loss function could be:

Combined Loss = Dice Loss + Focal Loss

- Jaccard Index (IoU):

Jaccard index, also known as Intersection over Union (IoU), is the area of the intersection over union of the predicted segmentation and the ground truth:

$$\mathrm{IoU} = \frac{Area\ of\ overlap}{Area\ of\ union} =$$



- F1-score:

The F-score combines precision and recall into a single metric, taking into account both measures. It is calculated using the harmonic mean of precision and recall. The formula for calculating the F-score (also known as F1 score) is as follows:

F1 = 2 * (precision * recall) / (precision + recall)

The F1 score ranges from 0 to 1, where 1 indicates perfect precision and recall, while 0 indicates poor performance. The F1 score is often used in segmentation tasks where the goal is to accurately identify and delineate objects or regions of interest within an image. By considering both precision and recall, the F1 score provides a balanced evaluation of the segmentation algorithm's performance. However, in some cases, researchers or practitioners might prioritize precision or recall more based on the specific requirements of their application, and they can use variations of the F-score that place different weights on precision and recall. For example, the F-beta score allows adjusting the importance of precision and recall by incorporating a weight parameter beta.

- **Fitting the model on our dataset**

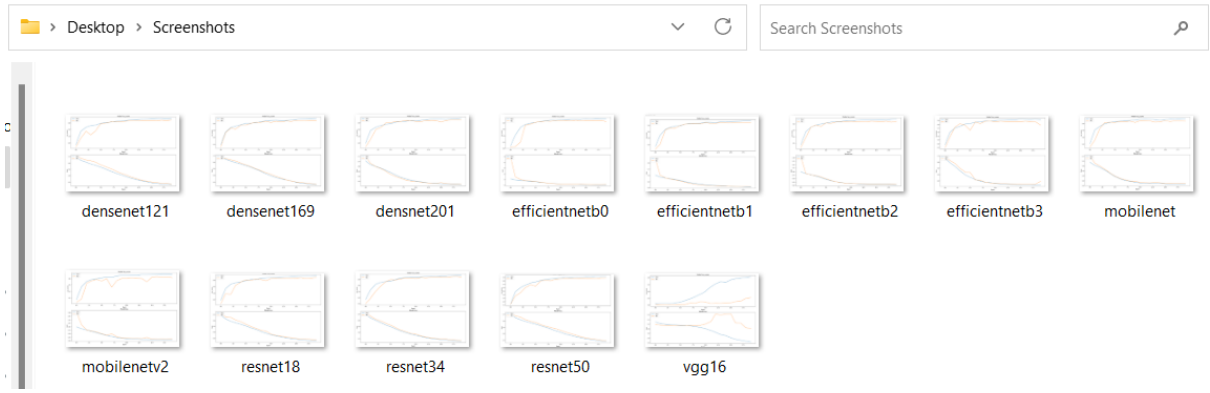
In [ ]:
```
# fitting the model
history=model.fit(x_train,
          y_train,
          batch_size=4,
          epochs=20,
          verbose=1,
          validation_data=(x_val, y_val))
```

- **Showing the training/validation loss and Iou score**

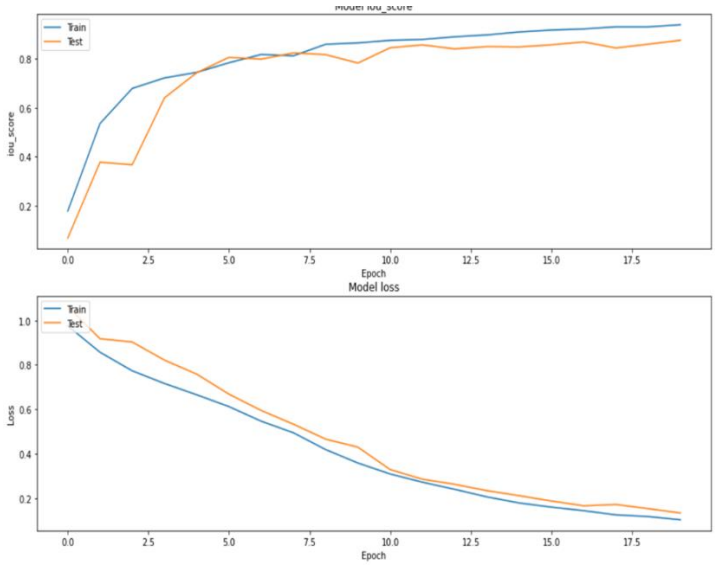
In [ ]:
```
# Plot training & validation iou_score values
plt.figure(figsize=(15, 10))
plt.subplot(211)
plt.plot(history.history['iou_score'])
plt.plot(history.history['val_iou_score'])
plt.title('Model iou_score')
plt.ylabel('iou_score')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'], loc='upper left')

# Plot training & validation loss values
plt.subplot(212)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'], loc='upper left')
plt.show()
```

We implemented with different types of back bones that you can observe as 2 charts:

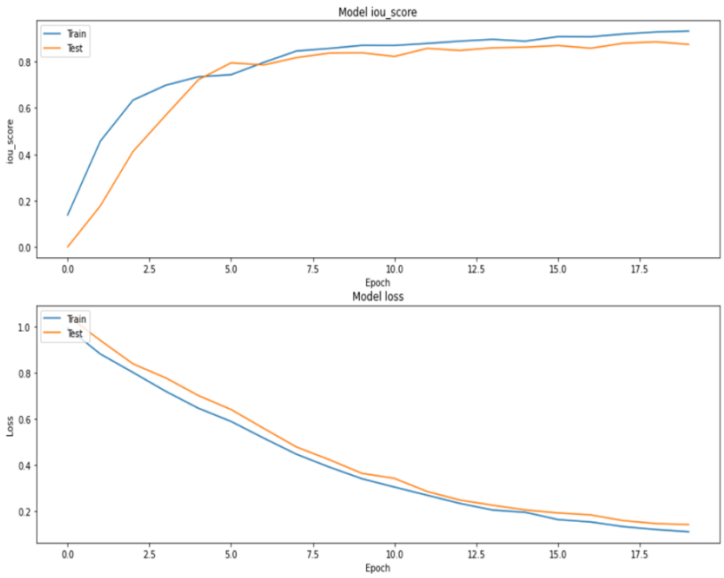
- training and validation loss
- training and validation Iou score



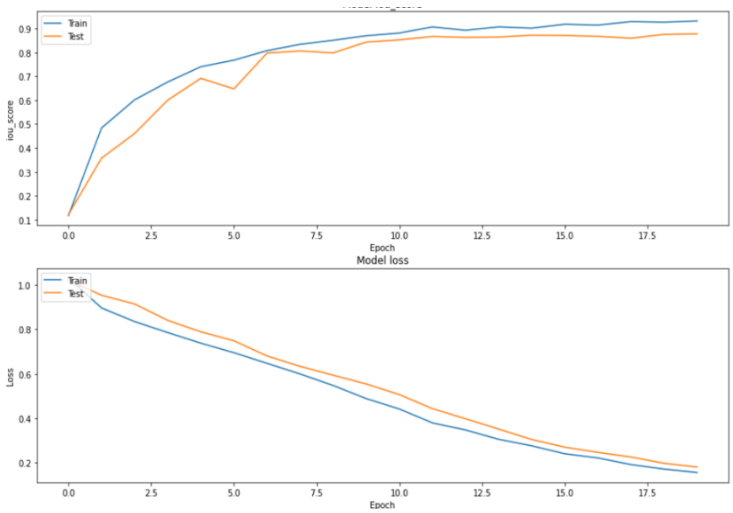
As might be expected, the training and validation losses should be decreased, and the training and validation should be increased during the training procedure. For example, you can observe the charts for 3 models of resnet18,34, and 50:



resnet18



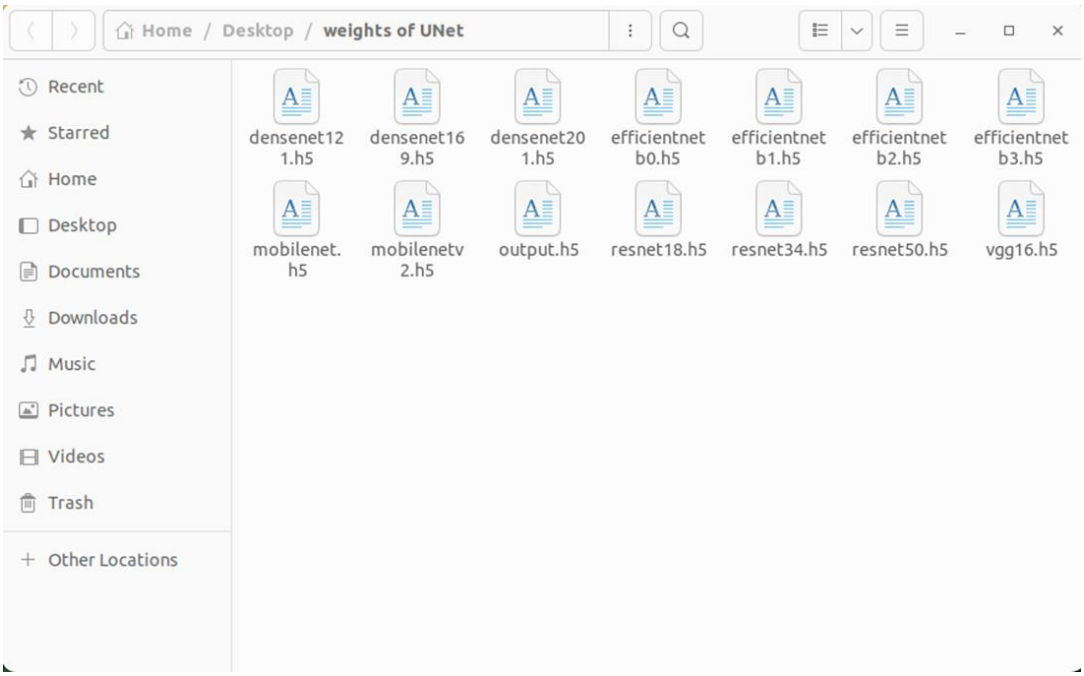
resnet34



resnet50

- **Evaluating different backbones based on loss and metrics**

THE WEIGHTS HAVE BEEN CRETAED BY DIFFERENT BACKBONES:



```python
model.save('resnet18.h5')
```

```python
model = keras.models.load_model('resnet18.h5',compile=False)

# define optomizer
LR= 0.0001
optim = keras.optimizers.Adam(LR)

# Segmentation models losses can be combined together by '+' and scaled by integer or float factor
dice_loss = sm.losses.DiceLoss()
focal_loss = sm.losses.BinaryFocalLoss()
total_loss = dice_loss + (1 * focal_loss)

metrics = [sm.metrics.IOUScore(threshold=0.5), sm.metrics.FScore(threshold=0.5)]

# compile keras model with defined optimozer, loss and metrics
model.compile(optim, total_loss, metrics)


score = model.evaluate(x_test, y_test, verbose=1)
```

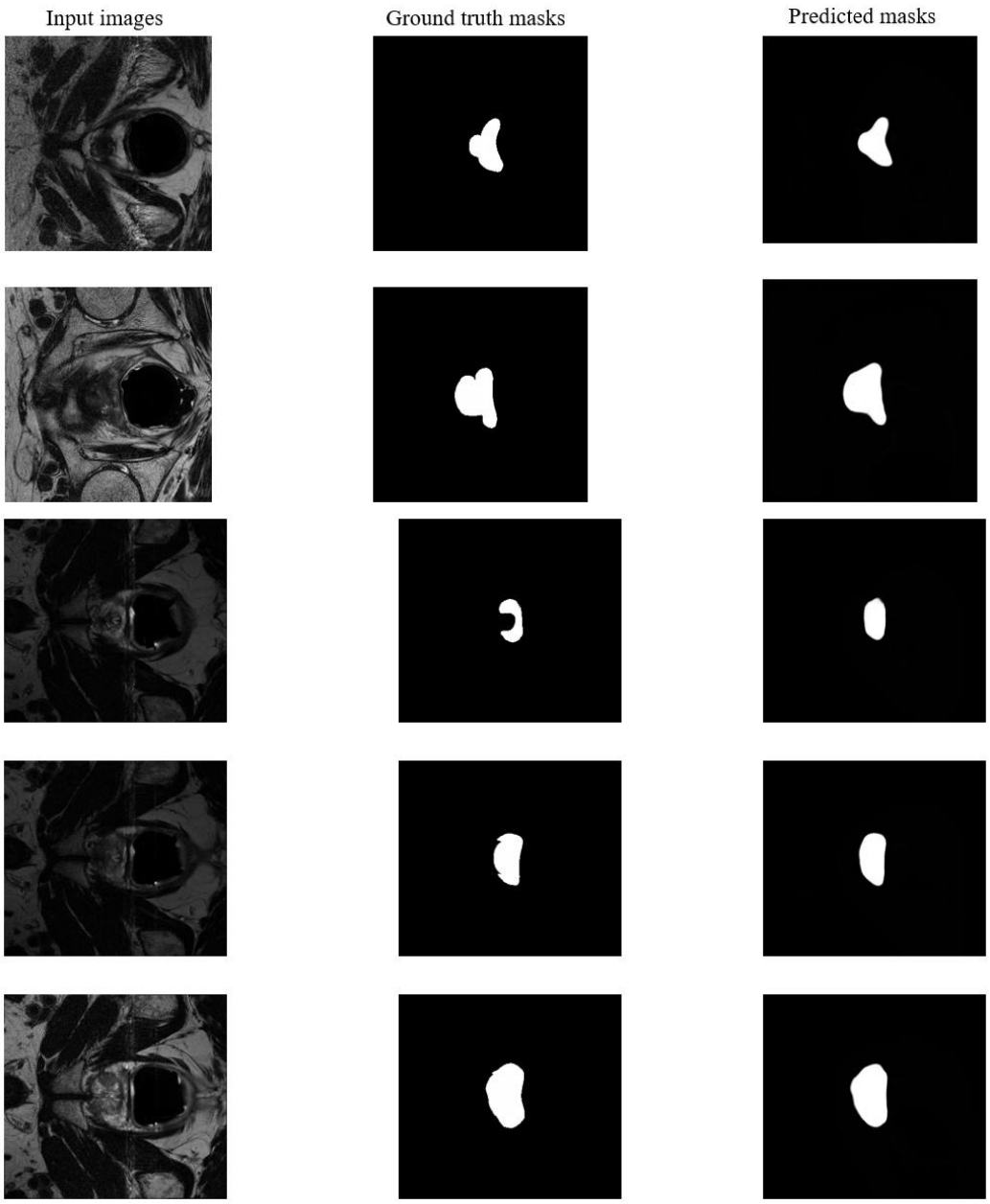
In addition, we have evaluated model on test data based on loss, Iou, and f1 scores. To concrete them, we have described the comparable data in the table below to show the model which has the best performance associated with different metrics. Based on this chart, the efficientnetb1 has the best performance in both Iou-Score with 0.8963 and F1-Score with 0.9453 in comparison with other backbones.

	loss	Iou-Score	F1-Score
resnet18	0.1295	0.8802	0.9363
resnet34	0.1340	0.8799	0.9361
resnet50	0.1755	0.8809	0.9367
densenet121	0.1330	0.8905	0.9421
densenet169	0.1346	0.8866	0.9399
densenet201	0.0837	0.8911	0.9424
efficientnetb0	0.1038	0.8593	0.9243
efficientnetb1	0.0874	0.8963	0.9453
efficientnetb2	0.0812	0.8889	0.9412
efficientnetb3	0.1740	0.7918	0.8838
Mobilenet	0.0912	0.8741	0.9328
mobilenetv2	0.1205	0.8383	0.9121

- **Testing on prostate images, to see the predicted masks in associated with real ground truth:**

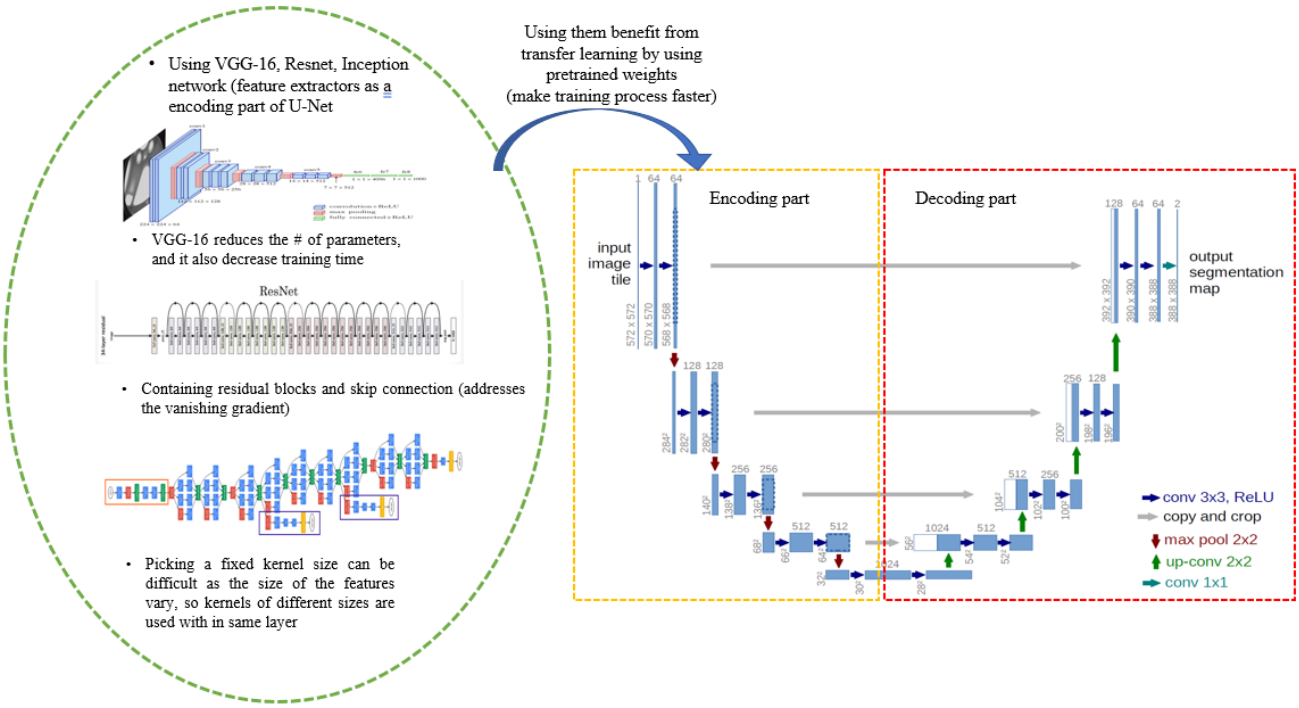
```
model = keras.models.load_model('resnet18.h5',compile=False)

#Test on a different image
#READ EXTERNAL IMAGE and preprocess it
test_img = cv2.imread('/home/behnoud/Desktop/Desktop/data/slices/img2/prostat4-slice009_z.png', cv2.IMREAD_COLOR)
test_img = cv2.resize(test_img, (SIZE_Y, SIZE_X))
test_img = cv2.cvtColor(test_img, cv2.COLOR_RGB2BGR)
test_img = np.expand_dims(test_img, axis=0)

# predicting trined model on test image
prediction = model.predict(test_img)

#View and Save segmented image
prediction_image = prediction.reshape(mask.shape)
plt.imshow(prediction_image, cmap='gray')
plt.imsave('/home/behnoud/Desktop/Desktop/data/slices/output.png', prediction_image, cmap='gray')
```

See some examples created by reset34 backbone in below:

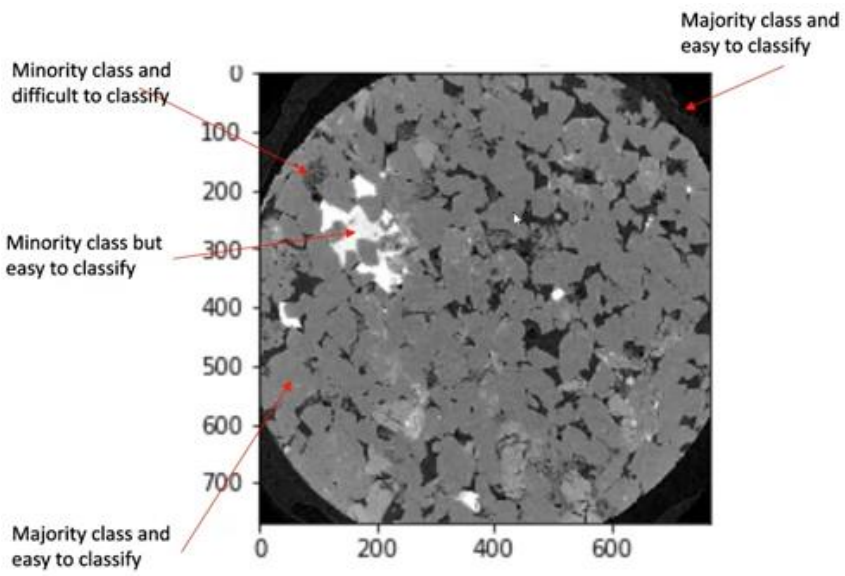


Conclusion and future works

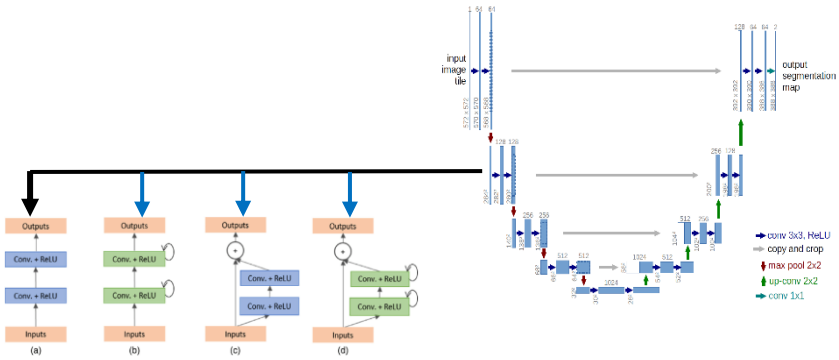
Instead of using from-scratched UNet model,we have used the transfer learning method including state-of-the-art pre-trained models on some huge dataset (like imagenet,…) leading to better convergence in training procedure, and less training time.



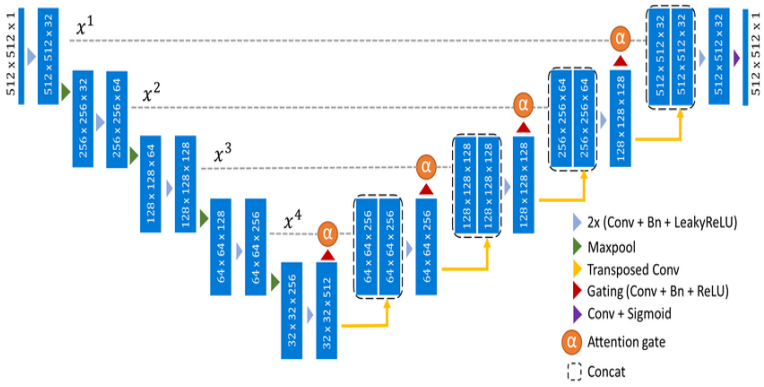
Moreover, in mostly time, in segmentation tasks, you face with imbalanced dataset (uneven dataset), which contains classes with various frequencies. In this situation, cross-entropy loss cannot handle unbalanced datasets. Even adding weight for different classes is not very effective. Thus, we have used focal loss and Iou loss both represent better results for unbalanced image segmentation. Specially focal loss change a gamma value to control importance of classes classified easier (or focusing on minority classes).



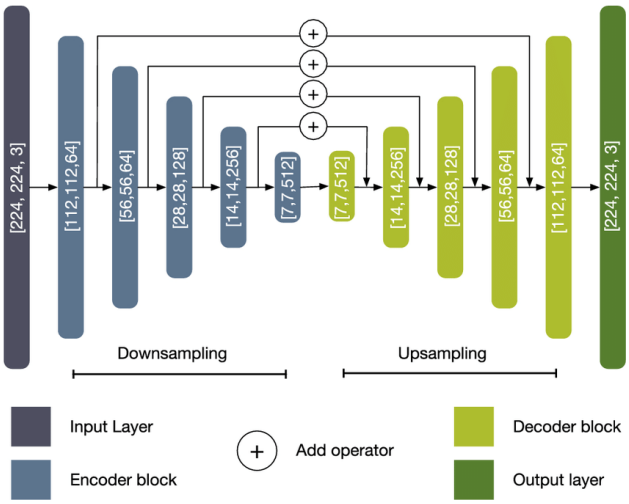
As a future works, to focus on context information of image, you can replace Conv block with variants of Conv blocks (Residual, Recurrent, and combination of them).



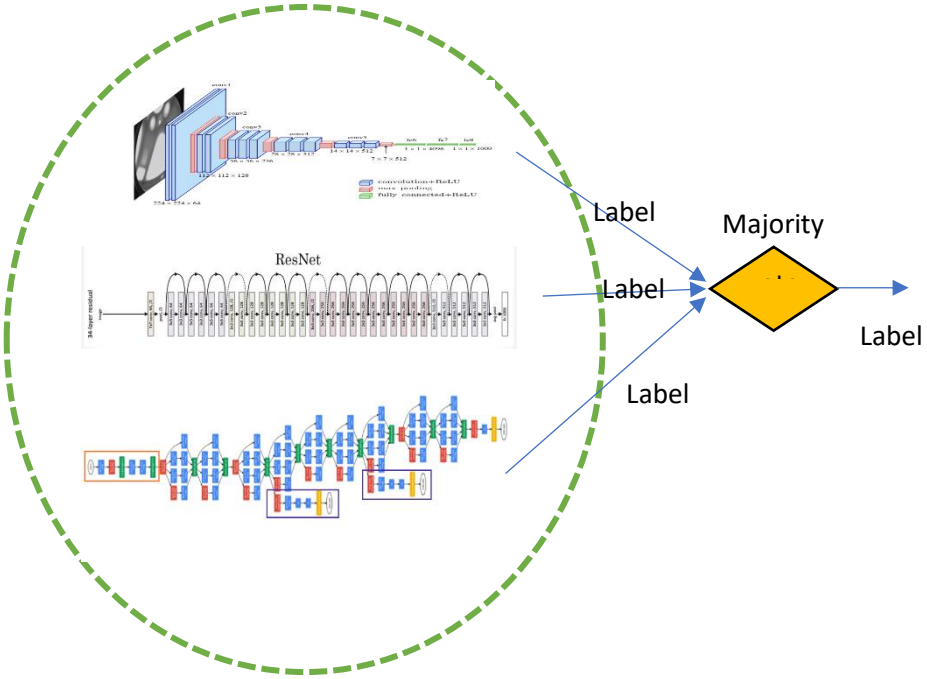
In addition, for Boosting the weak feature representation of initial layers when you add them with feature extracted from down layers (pay more attention on relevant region), you can use another structure called attention UNet.



To have good performance for real-time applications, the Link-net use add instead of concatenation (leading to decrease parameters and be faster).



Finally, to have better prediction, we can ensemble multiple trained networks (Resnet34+inceptionv3+VGG16) to improve semantic segmentation performance that Each model predicts a label for a pixel.



References

- Dataset: https://liuquande.github.io/SAML/
- https://medium.com/mlearning-ai/understanding-evaluation-metrics-in-medical-image-segmentation-d289a373a3f
- https://www.google.com/search?q=binary+focal+loss+formula&sxsrf=APwXEdcrWxYQvzQ2NuPZH6YzuRiatAteQw:1687560849142&source=lnms&tbm=isch&sa=X&ved=2ahUKEwj_tsyHvtr_AhWSt6QKHcWZDB0Q0pQJegQIBxAC&biw=1536&bih=714&dpr=1.25#imgrc=edGYrOOkiwSI3M
- https://www.youtube.com/@DennisMadsen
- https://www.youtube.com/@DigitalSreeni